

# Compiladores (CC3001) — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 2 (Aulas 1–4)

## Conteúdo

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>Aula 1 — Introdução e fases de um compilador</b>                              | <b>3</b>  |
| O que é um compilador . . . . .                                                  | 3         |
| Compiladores vs. interpretadores . . . . .                                       | 3         |
| Fases de um compilador . . . . .                                                 | 3         |
| Self-hosting e bootstrapping . . . . .                                           | 5         |
| <b>Aula 2 — Análise lexical</b>                                                  | <b>6</b>  |
| O que faz a análise lexical . . . . .                                            | 6         |
| Expressões regulares . . . . .                                                   | 6         |
| Autômatos finitos determinísticos (DFA) . . . . .                                | 7         |
| Autômatos finitos não-determinísticos (NFA) . . . . .                            | 8         |
| Conversão de NFA em DFA: construção de subconjuntos . . . . .                    | 10        |
| Minimização de autômatos . . . . .                                               | 12        |
| <b>Aula 3 — Geradores de analisadores lexicais</b>                               | <b>14</b> |
| Implementação direta de um DFA em C . . . . .                                    | 14        |
| Alex (gerador para Haskell) . . . . .                                            | 15        |
| Flex (gerador para C) . . . . .                                                  | 15        |
| Longest match e first match . . . . .                                            | 16        |
| Comparação rápida Alex vs. Flex . . . . .                                        | 17        |
| Ligação com a prática (Folha laboratorial 2 — <code>scanner-c</code> ) . . . . . | 17        |
| <b>Aula 4 — Análise sintática e gramáticas independentes de contexto</b>         | <b>18</b> |
| O que é a análise sintática . . . . .                                            | 18        |
| Gramáticas independentes de contexto . . . . .                                   | 18        |
| Derivações . . . . .                                                             | 19        |
| Linguagem descrita por uma gramática . . . . .                                   | 19        |
| Árvores sintáticas . . . . .                                                     | 20        |
| Gramáticas ambíguas . . . . .                                                    | 21        |
| Exemplo central: expressões aritméticas . . . . .                                | 23        |
| Eliminar ambiguidade: associatividade e precedência . . . . .                    | 23        |
| O problema do “dangling else” . . . . .                                          | 24        |

## Aula 1 — Introdução e fases de um compilador

### O que é um compilador

Um **compilador** é um tradutor de programas: recebe um programa escrito numa linguagem (a *linguagem fonte*) e produz um programa equivalente noutra linguagem (a *linguagem destino*). Normalmente traduz de uma linguagem de **alto nível** (ex: C, Java) para uma de **baixo nível** (código máquina, ou uma representação intermédia como bytecode JVM). É a principal técnica usada para implementar linguagens de programação — GCC (C/C++ → código máquina), Javac (Java → bytecode JVM), GHC (Haskell → código máquina) e Scalac (Scala → bytecode JVM) são todos compiladores.

#### Exemplo resolvido

Um caso interessante é **elm**, que compila Elm para **JavaScript** — o destino nem sempre é código máquina; pode ser outra linguagem de alto nível (a isto também se chama *transpilação*, mas o mecanismo interno é o mesmo de um compilador).

### Compiladores vs. interpretadores

Em vez de traduzir o programa todo antes de o correr, um **interpretador** faz a análise lexical e sintática da mesma forma que um compilador, mas depois **executa diretamente** o programa a partir da árvore sintática (ou, em alternativa, gera e executa código intermédio à medida que avança, sem produzir um executável final).

#### Definição — Compilação vs. interpretação

**Compilação:** traduz o programa fonte inteiro para outra linguagem *antes* de o executar, produzindo um artefacto (executável) reutilizável. **Interpretação:** analisa e executa o programa diretamente, sem produzir um executável separado.

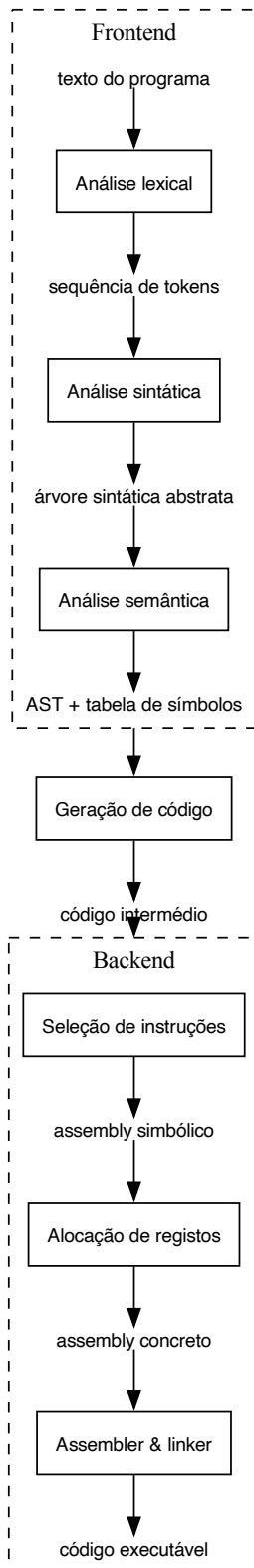
|               | Vantagem                                                                                                                                                  |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interpretação | Mais simples de implementar; mais fácil suportar várias arquiteturas; permite REPL ( <i>read-eval-print-loop</i> ), útil para desenvolvimento interativo. |
| Compilação    | O código compilado é normalmente mais eficiente; o executável pode ser distribuído sem depender do compilador.                                            |

#### Atenção

Nota adicional: muitas implementações reais **combinam** as duas técnicas (ex: GHC pode compilar ou interpretar Haskell; OCaml tem `ocaml` — interpretador/REPL — e `ocamlopt` — compilador). Não penses nelas como mutuamente exclusivas.

### Fases de um compilador

Um compilador divide-se tradicionalmente em duas grandes partes, ligadas por uma **representação intermédia** (código intermédio) que é independente da linguagem fonte e da máquina destino:



**Definição — Frontend / Backend**

**Frontend** (depende da linguagem fonte, não da máquina destino): análise lexical → análise sintática → análise semântica → AST + tabela de símbolos.

**Backend** (depende da máquina destino, não da linguagem fonte): seleção de instruções → alocação de registos → assembler & linker.

Entre os dois: **geração de código intermédio**, que não depende nem da linguagem fonte nem da máquina destino.

Esta separação torna o compilador mais simples e permite **reutilizar componentes**: o *backend* do GCC serve para compilar C, C++ ou Objective-C; o *backend* LLVM é partilhado por Clang, Swift, Rust e (opcionalmente) GHC. Um mesmo *frontend* também pode, em teoria, ser usado com vários *backends* diferentes.

Este semestre o foco está sobretudo no **frontend** — é aí que entram a análise lexical (Aulas 2–3, abaixo) e a análise sintática.

**Self-hosting e bootstrapping****Definição — Self-hosting**

Um compilador é **self-hosted** quando está escrito na própria linguagem que compila. Exemplos: GCC é escrito em C, Javac em Java, GHC em Haskell, Rustc em Rust.

Self-hosting é normalmente visto como prova de maturidade da linguagem e das suas ferramentas (“*eat your own dog food*”) e facilita a comunicação entre programadores do próprio compilador (só precisam de saber uma linguagem). O problema óbvio é: **se o compilador de X está escrito em X, o que compilou a primeira versão?**

**Atenção — Bootstrapping**

Resolve-se por **bootstrapping**: escreve-se uma primeira versão do compilador numa *outra* linguagem, usa-se essa versão para compilar uma segunda versão já escrita na linguagem-alvo, e repete-se o processo até o compilador conseguir compilar-se a si próprio.

- **C**: o primeiro compilador de C (1973) foi baseado no compilador de B (uma linguagem anterior); o primeiro compilador de B tinha sido escrito em TMG (uma linguagem do sistema Multics). O compilador foi depois reescrito em B, por estágios, até ser capaz de compilar a si próprio.
- **Rust**: o primeiro compilador (2006–2010) foi escrito em OCaml; a partir de 2010 começou a reescrita em Rust (ainda compilada pelo compilador OCaml); desde 2011 o compilador de Rust é *self-hosted*.

## Aula 2 — Análise lexical

### O que faz a análise lexical

O **léxico** de uma linguagem de programação é o conjunto de símbolos e palavras usados para compor programas — os *tokens*. Há várias categorias típicas: **identificadores** (nomes de variáveis/funções), **literais** (números, caracteres, cadeias), **palavras reservadas** (`if`, `else`, `while`, ...) e **operadores** (`+`, `*`, `=`, ...).

Um **analisador lexical** (também chamado *lexer*, *scanner* ou *tokenizer*) decompõe o texto de entrada numa sequência de *tokens*. Esta decomposição facilita a análise sintática seguinte — o *parser* trabalha sobre uma lista de *tokens* já classificados, não sobre texto em bruto carácter a carácter.

#### Definição — Representação de um token

Cada *token* é identificado por uma **etiqueta** (*token type*), e alguns tipos têm também um **valor** associado:

| Entrada           | Etiqueta | Valor  |
|-------------------|----------|--------|
| <code>if</code>   | IF       | —      |
| <code>,</code>    | COMMA    | —      |
| <code>!=</code>   | NOTEQ    | —      |
| <code>main</code> | ID       | "main" |
| <code>71</code>   | NUM      | 71     |
| <code>.5</code>   | REAL     | 0.5    |

#### Exemplo resolvido

Para o código C `float match0(char *s) { if (!strcmp(s, "0.0", 3)) return 0.0; },` o analisador lexical produz a sequência:

FLOAT, ID("match0"), LPAREN, CHAR, STAR, ID("s"), RPAREN, LBRACE, IF, LPAREN, BANG, ID("strcmp"), LPAREN, ID("s"), COMMA, STRING("0.0"), COMMA, NUM(3), RPAREN, RPAREN, RETURN, REAL(0.0), SEMI, RBRACE, EOF

Repara que **espaços, mudanças de linha, tabulações e comentários não geram tokens** — são consumidos e descartados pelo analisador (servem só para separar tokens). Um token EOF marca explicitamente o fim da entrada.

### Expressões regulares

Escrever um analisador lexical à mão, carácter a carácter (com um `switch` gigante, por exemplo), é simples mas repetitivo, difícil de manter e fácil de errar (ex: é preciso ter cuidado com a ordem das condições — "`if`" tem de ser reconhecido como palavra reservada e não como identificador). A alternativa é **descrever os tokens de forma declarativa** usando expressões regulares, e depois gerar automaticamente o analisador a partir dessa descrição — é isso que os geradores de analisadores (Aula 3) fazem.

#### Definição — Expressões regulares (REs)

Dado um alfabeto  $\Sigma$ , uma **linguagem** é um subconjunto  $L \subseteq \Sigma^*$  de palavras formadas a partir de  $\Sigma$ . As expressões regulares constroem-se assim:

- $a$  — uma letra  $a \in \Sigma$  (reconhece só a palavra " $a$ ")
- $\varepsilon$  — a palavra vazia
- $M \mid N$  — **união**: reconhece o que  $M$  ou  $N$  reconhecem
- $M \cdot N$  — **concatenação**: palavras  $u \cdot v$  tais que  $u \in L(M)$  e  $v \in L(N)$
- $M^*$  — **repetição** (fecho de Kleene): concatenação de zero ou mais palavras de  $M$

### Definição — Convenções e abreviaturas

O símbolo de concatenação escreve-se muitas vezes de forma implícita:  $abc = (a \cdot b) \cdot c$  (não é ambíguo, por associatividade). A prioridade dos operadores é **repetição** > **concatenação** > **união**, ou seja  $ab|c = (ab)|c$  e  $abb^* = ab(b^*)$ .

Abreviaturas comuns:

| Notação              | Significado                                                   |
|----------------------|---------------------------------------------------------------|
| $M?$                 | opcional: $M \mid \varepsilon$                                |
| $M^+$                | uma ou mais vezes: $M \cdot M^*$                              |
| $[x_1x_2 \dots x_n]$ | alternativa entre caracteres: $x_1 \  x_2 \  \dots \  x_n$    |
| $[a-z]$              | intervalo de caracteres contíguos                             |
| $[^a-z]$             | complementar: qualquer carater fora do intervalo              |
| $\cdot$              | qualquer carater exceto mudança de linha                      |
| "a+"                 | cadeia literal (entre aspas, sem interpretar + como operador) |

### Exemplo resolvido

Descrever os *tokens* de um pequeno analisador em C:

```
if                palavra reservada (IF)
[_a-zA-Z][_a-zA-Z0-9]*  identificadores (ID)
[0-9]+            número inteiro (NUM)
([0-9]+|" [0-9]*")|([0-9]*"."[0-9]+)  número fracionário (REAL)
//.*             comentário até ao final da linha
```

Note-se que `[_a-zA-Z][_a-zA-Z0-9]*` também reconheceria `if` — por isso a ordem/prioridade importa: as regras dos geradores (Aula 3) tratam este caso com a regra do “*first match*”, colocando `if` antes do padrão de identificador.

### Atenção

**Nota adicional** (não estava explícito nos slides, mas é a razão pela qual o Exercício 2(c) do laboratório avisa que os comentários multi-linha “são mais difíceis do que parece”): à primeira vista pareceria bastar `"/" . .* . "/"` para reconhecer um comentário `/* ... */`. O problema é que `.` costuma **incluir tudo menos a mudança de linha**, e `.*` é *greedy* — tentará consumir o máximo de caracteres possível. Numa entrada como `/* a */ x /* b */`, essa expressão reconheceria a linha inteira como **um único** comentário (desde o primeiro `/*` até ao **último** `*/`), em vez de dois comentários separados. A técnica geral para “parar no primeiro terminador” é excluir explicitamente o carácter do terminador do meio da expressão, por exemplo (usando `<!--/-->` como ilustração diferente do exercício, para não to resolver): um comentário `<!-- conteúdo -->` descreve-se como `"<!--" . ([^-" | "-" [^-" ])* . "-->"` — “qualquer carácter que não seja `-`, ou um `-` que não seja seguido de outro `-`”, até fechar com `-->`. A mesma ideia aplica-se ao `/* ... */` do exercício, adaptada ao terminador `*/`.

## Autómatos finitos determinísticos (DFA)

As expressões regulares são **descrições declarativas**: dizem o que reconhecer, não como. Para implementar um analisador precisamos de um **modelo operacional** — um autómato finito.

**Definição — DFA**

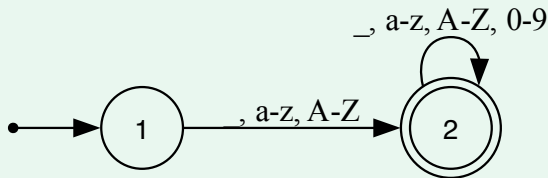
Um **autômato finito determinístico** é um quintuplo  $(\Sigma, Q, q_0, F, \delta)$ :

- $\Sigma$  — alfabeto
- $Q$  — conjunto finito de estados
- $q_0 \in Q$  — estado inicial (único)
- $F \subseteq Q$  — conjunto de estados finais (de aceitação)
- $\delta \subseteq Q \times \Sigma \times Q$  — transições, **determinísticas**: para cada  $q \in Q$  e  $\sigma \in \Sigma$  existe **no máximo um**  $q'$  tal que  $(q, \sigma, q') \in \delta$ . Também se pode ver  $\delta$  como uma função parcial  $q' = \delta(q, \sigma)$ .

Uma palavra é **aceite** se, partindo de  $q_0$  e seguindo as transições para cada símbolo da palavra, se terminar num estado de  $F$ .

**Exemplo resolvido**

DFA equivalente à expressão regular de identificadores `[_a-zA-Z][_a-zA-Z0-9]*`:



Formalmente:  $\Sigma =$  caracteres ASCII,  $Q = \{1, 2\}$ ,  $q_0 = 1$ ,  $F = \{2\}$ , e  $\delta$  contém  $(1, \ell, 2)$  para toda a letra/underscore  $\ell$ , e  $(2, \ell, 2)$ ,  $(2, d, 2)$  para toda a letra/underscore  $\ell$  e dígito  $d$ .

Percorrendo `x1`:  $1 \xrightarrow{x} 2 \xrightarrow{1} 2$ . Como  $2 \in F$ , a palavra é **aceite**.

**Autômatos finitos não-determinísticos (NFA)****Definição — NFA**

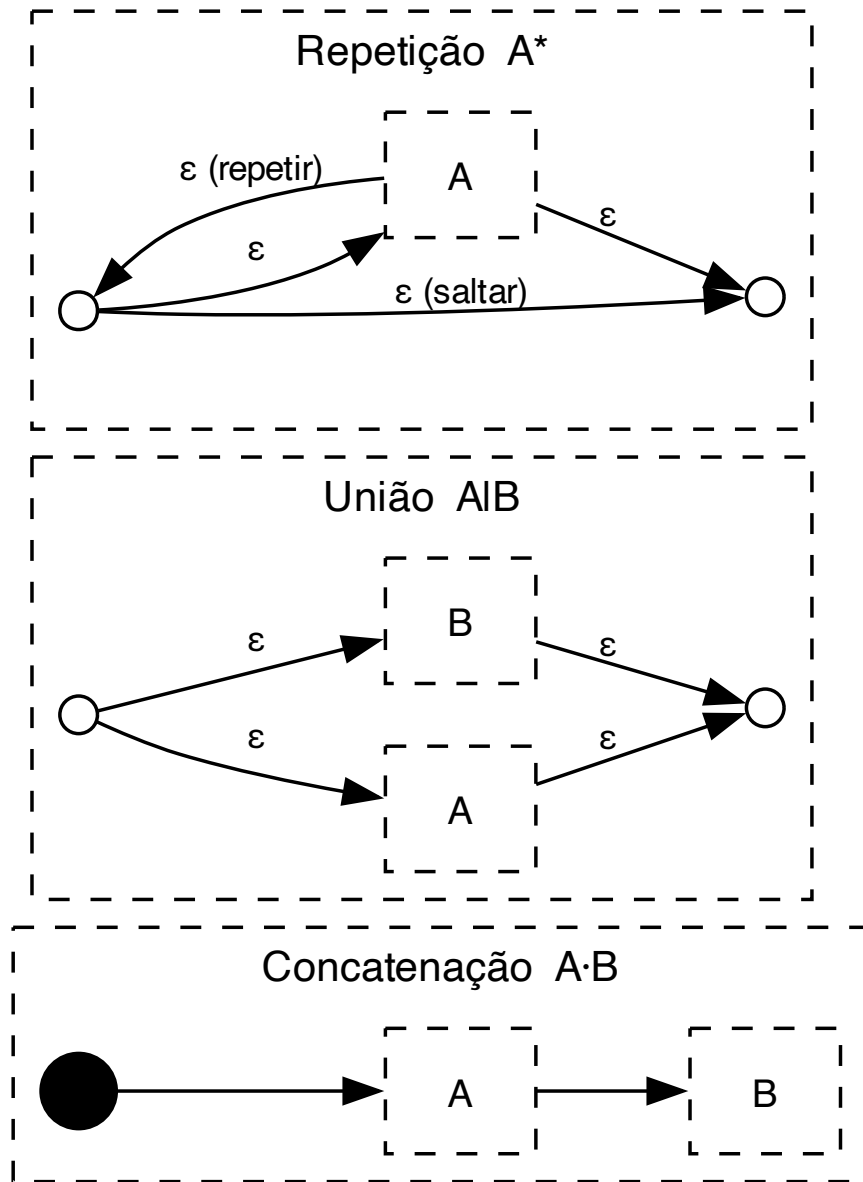
Um **NFA** tem a mesma estrutura que um DFA, mas as transições podem ser **não-determinísticas**:  $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$ . Isto permite duas coisas que um DFA não tem:

- **várias transições** com o mesmo símbolo a partir do mesmo estado (ex: de 1, duas transições distintas com `a`, para 2 e para 3);
- **transições- $\varepsilon$** , que mudam de estado **sem consumir** nenhum símbolo de entrada.

Um NFA não se pode implementar diretamente da mesma forma que um DFA (nunca sabemos, num dado momento, “em que estado estamos” — pode haver vários possíveis ao mesmo tempo, e as transições- $\varepsilon$  complicam ainda mais a leitura carácter a carácter). Por isso o percurso normal é: **expressão regular**  $\rightarrow$  **NFA** (fácil)  $\rightarrow$  **DFA equivalente** (mais complexo, mas implementável diretamente).

**Construção de Thompson: de expressão regular a NFA**

A construção de Thompson traduz uma expressão regular num NFA de forma composicional: cada sub-expressão vira um **fragmento** de autômato com uma única entrada e uma única saída, e os fragmentos combinam-se seguindo a estrutura da expressão.



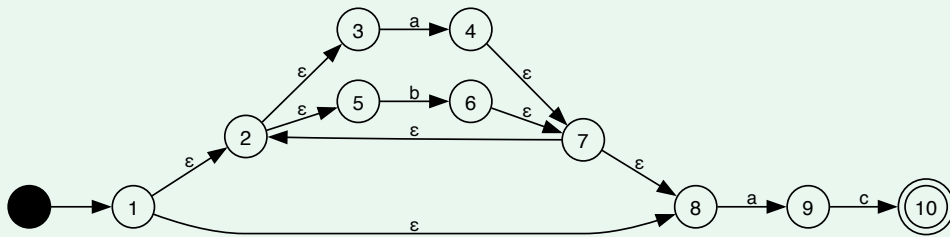
### Definição — Regras de Thompson

- **Símbolo**  $\sigma \in \Sigma$ : um fragmento com uma única transição rotulada  $\sigma$  da entrada para a saída (e igual para  $\varepsilon$ ).
- **Concatenação**  $A \cdot B$ : a saída do fragmento de  $A$  liga-se diretamente à entrada do fragmento de  $B$ .
- **União**  $A \mid B$ : um novo estado de entrada liga-se, por  $\varepsilon$ , às entradas de  $A$  e de  $B$ ; as saídas de  $A$  e de  $B$  ligam-se, por  $\varepsilon$ , a um novo estado de saída comum.
- **Repetição**  $A^*$ : um novo par entrada/saída; a entrada liga-se por  $\varepsilon$  à entrada de  $A$  (para iterar) e diretamente à saída (para “saltar” zero repetições); a saída de  $A$  liga-se por  $\varepsilon$  de volta à sua própria entrada (para repetir) e à nova saída (para terminar).

### Exemplo resolvido

Construir o NFA para  $(a \mid b)^* \cdot a \cdot c$  (o exemplo da aula), passo a passo:

1. Fragmento para **a** (estados 3→4) e fragmento para **b** (estados 5→6).
2. **União**  $a \mid b$ : novo estado 2 (entrada,  $\varepsilon$  para 3 e para 5) e novo estado 7 (saída,  $\varepsilon$  a partir de 4 e de 6).
3. **Repetição**  $(a \mid b)^*$ : novo par 1 (entrada) / 8 (saída);  $1 \rightarrow 2$  (entrar no corpo),  $1 \rightarrow 8$  (saltar, zero repetições),  $7 \rightarrow 2$  (repetir),  $7 \rightarrow 8$  (terminar o ciclo).
4. **Concatenação** com **a**: a saída do  $*$  (estado 8) liga-se por **a** a um novo estado 9.
5. **Concatenação** com **c**: o estado 9 liga-se por **c** ao estado final 10.



## Conversão de NFA em DFA: construção de subconjuntos

### Definição — closure e construção de subconjuntos

Seja  $\mathcal{A} = (\Sigma, Q, q_0, F, \delta)$  um NFA. Define-se

$$\text{closure}(S) = \{ q' \in Q : \text{existe } q \in S \text{ e um caminho de transições-}\varepsilon \text{ entre } q \text{ e } q' \}$$

(ou seja, todos os estados alcançáveis a partir de  $S$  **sem consumir nenhum símbolo**). O NFA transforma-se no DFA equivalente  $\mathcal{A}' = (\Sigma, 2^Q, S_0, F', \delta')$ , onde **cada estado do DFA é um conjunto de estados do NFA**:

$$S_0 = \text{closure}(\{q_0\}) \quad F' = \{ S \subseteq Q : S \cap F \neq \emptyset \} \quad \delta'(S, \sigma) = \text{closure}(\{ q' : q \in S \wedge (q, \sigma, q') \in \delta \})$$

Como  $\mathcal{A}'$  é determinístico,  $\delta'$  pode ver-se como função.

## Exemplo resolvido

Aplicando a construção de subconjuntos ao NFA de  $(a \mid b)^*ac$  construído acima. Transições- $\varepsilon$  do NFA (para calcular *closures*):  $1 \rightarrow 2$ ,  $1 \rightarrow 8$ ,  $2 \rightarrow 3$ ,  $2 \rightarrow 5$ ,  $4 \rightarrow 7$ ,  $6 \rightarrow 7$ ,  $7 \rightarrow 2$ ,  $7 \rightarrow 8$ . Transições com símbolo:  $3 \xrightarrow{a} 4$ ,  $5 \xrightarrow{b} 6$ ,  $8 \xrightarrow{a} 9$ ,  $9 \xrightarrow{c} 10$ .

**Estado inicial:**  $S_0 = \text{closure}(\{1\}) = \{1, 2, 3, 5, 8\}$  — partindo de 1, seguimos só  $\varepsilon$ :  $1 \rightarrow 2$ ,  $2 \rightarrow 3$ ,  $2 \rightarrow 5$ ,  $1 \rightarrow 8$ ; nenhum destes  $(2, 3, 5, 8)$  tem mais  $\varepsilon$ -saídas, por isso o *closure* para aqui.

Calculamos agora, **estado a estado**, para onde vai  $S_0$  com cada símbolo:

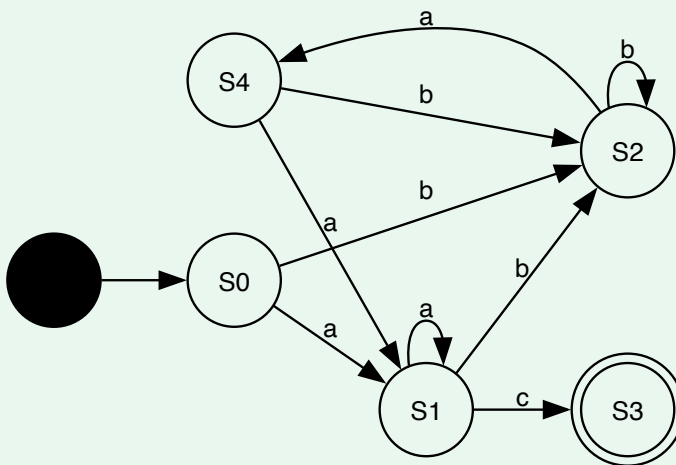
- $S_0 = \{1, 2, 3, 5, 8\}$  **com a**: só os estados 3 e 8 (de  $S_0$ ) têm transição por **a** ( $3 \xrightarrow{a} 4$  e  $8 \xrightarrow{a} 9$ ), logo  $\text{move} = \{4, 9\}$ .  $\text{closure}(\{4, 9\})$ : de 4,  $\varepsilon$  para 7, depois  $7 \rightarrow 2$  e  $7 \rightarrow 8$ , depois  $2 \rightarrow 3$  e  $2 \rightarrow 5$ ; de 9 não há mais  $\varepsilon$ . Total:  $\{2, 3, 4, 5, 7, 8, 9\}$  — como este conjunto ainda não tinha aparecido, chamamos-lhe  $S_1$ .
- $S_0$  **com b**: só o estado 5 tem transição por **b** ( $5 \xrightarrow{b} 6$ ), logo  $\text{move} = \{6\}$ .  $\text{closure}(\{6\})$ :  $6 \rightarrow 7$ , depois  $7 \rightarrow 2$ ,  $7 \rightarrow 8$ , depois  $2 \rightarrow 3$ ,  $2 \rightarrow 5$ . Total:  $\{2, 3, 5, 6, 7, 8\}$  — novo, chamamos-lhe  $S_2$ .
- $S_0$  **com c**: nenhum estado de  $S_0 = \{1, 2, 3, 5, 8\}$  tem transição por **c** (só o estado 9 tem, e  $9 \notin S_0$ ) — **sem transição definida**.

## Exemplo resolvido

Continuando a partir de  $S_1 = \{2, 3, 4, 5, 7, 8, 9\}$  e  $S_2 = \{2, 3, 5, 6, 7, 8\}$ :

- $S_1$  **com a**: estados de  $S_1$  com transição a: 3 e 8 (os mesmos de antes, ambos  $\in S_1$ )  $\Rightarrow$  move =  $\{4, 9\}$ , closure =  $\{2, 3, 4, 5, 7, 8, 9\} = S_1$  — **fica no mesmo estado** (*self-loop* em a).
- $S_1$  **com b**: estado 5  $\in S_1$  tem transição b  $\Rightarrow$  move =  $\{6\}$ , closure =  $\{2, 3, 5, 6, 7, 8\} = S_2$ .
- $S_1$  **com c**: estado 9  $\in S_1$  tem transição c ( $9 \xrightarrow{c} 10$ )  $\Rightarrow$  move =  $\{10\}$ , closure( $\{10\}$ ) =  $\{10\}$  (o 10 não tem  $\varepsilon$ -saídas) — novo, chamamos-lhe  $S_3$ . Como 10 é o estado final do NFA,  $S_3 \in F'$ .
- $S_2$  **com a**: só o estado 3  $\in S_2$  tem transição a (o 8 **não** está em  $S_2$ , por isso desta vez só contamos o 3)  $\Rightarrow$  move =  $\{4\}$ , closure( $\{4\}$ ):  $4 \rightarrow 7, 7 \rightarrow 2, 7 \rightarrow 8, 2 \rightarrow 3, 2 \rightarrow 5$ . Total:  $\{2, 3, 4, 5, 7, 8\}$  — novo, chamamos-lhe  $S_4$ .
- $S_2$  **com b**: estado 5  $\in S_2 \Rightarrow$  move =  $\{6\}$ , closure =  $\{2, 3, 5, 6, 7, 8\} = S_2$  — *self-loop* em b.
- $S_2$  **com c**: nenhum estado de  $S_2$  tem transição c — sem transição.
- $S_4 = \{2, 3, 4, 5, 7, 8\}$  **com a**: estados 3 e 8  $\in S_4 \Rightarrow$  move =  $\{4, 9\}$ , closure =  $\{2, 3, 4, 5, 7, 8, 9\} = S_1$  (já conhecido).
- $S_4$  **com b**: estado 5  $\in S_4 \Rightarrow$  move =  $\{6\}$ , closure =  $\{2, 3, 5, 6, 7, 8\} = S_2$  (já conhecido).
- $S_4$  **com c**: nenhum estado de  $S_4$  tem transição c — sem transição.

Todas as transições de  $S_4$  vão para estados já conhecidos ( $S_1, S_2$ ) — **não aparece nenhum estado novo**, o processo termina. Resultado final: 5 estados  $S_0$  a  $S_4$ , com  $F' = \{S_3\}$  (único conjunto que contém o estado final 10 do NFA):



## Minimização de autómatos

O DFA obtido pela construção de subconjuntos pode ter **estados redundantes** — estados diferentes (como conjuntos) mas com exatamente o mesmo comportamento (as mesmas transições de saída para todo o alfabeto, e o mesmo estatuto de aceitação). Podemos “fundir” esses estados equivalentes e obter um **autômato mínimo** equivalente. Os geradores de analisadores lexicais (Alex, Flex — Aula 3) fazem esta minimização automaticamente.

**Frequente em exame**

**Sai frequentemente em exame:** reconhecer estados equivalentes. No exemplo acima, repara que  $S_0 = \{1, 2, 3, 5, 8\}$  e  $S_4 = \{2, 3, 4, 5, 7, 8\}$  têm exatamente as mesmas transições ( $\xrightarrow{a} S_1$ ,  $\xrightarrow{b} S_2$ , sem transição em c) — a diferença entre os conjuntos (1 vs.  $\{4, 7\}$ ) não afeta o comportamento, porque nenhum desses estados extra tem transições que consomem símbolos. Logo  $S_0 \equiv S_4$ : seriam fundidos na minimização, reduzindo o autômato de 5 para 4 estados. Este é exatamente o tipo de equivalência ( $S_0 \equiv S_2$  no exemplo da aula) que costuma aparecer nos exercícios de exame sobre minimização.

## Aula 3 — Geradores de analisadores lexicais

Recapitulando o percurso: **expressão regular** → **NFA (Thompson)** → **DFA (subconjuntos)** → **implementação**. Esta aula cobre o último passo: como implementar um analisador a partir de um DFA, e como **gerar** esse analisador automaticamente a partir das expressões regulares (sem passar manualmente pelos passos intermédios).

### Implementação direta de um DFA em C

Um DFA pequeno pode implementar-se com uma **tabela de transições** (`delta[estado][símbolo]`) e um ciclo que lê caracteres e segue as transições:

#### Exemplo resolvido

Para o DFA de identificadores (secção anterior), com estado 0 reservado para representar “erro” (ausência de transição válida) e estado final 2:

```
#define STATES 3    // estados 0 (erro), 1, 2
#define SYMBOLS 128 // códigos ASCII
const int delta[STATES][SYMBOLS] = {
    /* estado 0 */ { 0, ..., 0, ... },    // erro é absorvente
    /* estado 1 */ { 0, ..., 2, ..., 2, ... 0 }, // letra/_ -> 2
    /* estado 2 */ { 0, ..., 2, ..., 2, ... 0 } }; // letra/_/dígitto -> 2

int state = 1;    // estado inicial
int c;
while ((c = getchar()) != EOF) {
    state = delta[state][c];
    if (state == 0) return ERROR; // rejeita
    if (FINAL(state)) return ACCEPT; // aceita (aqui, FINAL(state) é state==2)
}
return ERROR; // fim do input sem atingir estado final
```

Traçando a execução com a entrada “x1”:

| Passo | c lido    | delta[state][c]   | novo state | ação                                                                              |
|-------|-----------|-------------------|------------|-----------------------------------------------------------------------------------|
| 1     | x (letra) | delta[1]['x'] = 2 | 2          | state != 0 (não rejeita); FINAL(2) é verdade → <b>return ACCEPT imediatamente</b> |

O ciclo **nunca chega a ler o 1!** Isto revela uma limitação importante deste pseudo-código, que vale a pena perceber bem: ele aceita assim que atinge **qualquer** estado final, não espera para ver se consumindo mais caracteres continuaria num estado válido (não implementa *longest match*). Para este DFA de identificadores em particular isto até não causa erro de classificação (qualquer prolongamento de um identificador continua a ser um identificador válido), mas é exatamente o motivo por que os analisadores lexicais reais (Aula 3, secção “Longest match e first match”, abaixo) têm de **guardar a posição do último estado final visto e continuar a tentar consumir mais caracteres**, só decidindo aceitar quando deixar de haver transição possível — este pseudo-código simplificado dos slides ilustra a ideia da tabela de transições, mas não implementa essa regra.

Isto funciona como ilustração da tabela de transições, mas é **específico de um único DFA** — cada novo token exigiria escrever/combinar mais tabelas à mão. É exatamente este trabalho mecânico que os geradores (abaixo) automatizam a partir de expressões regulares.

## Alex (gerador para Haskell)

### Definição — Alex

O **Alex** é um gerador de analisadores lexicais para Haskell. Recebe um ficheiro de descrição (ex: `Lexer.x`) com regras “expressão regular  $\rightarrow$  ação em Haskell” e definições auxiliares, e produz um ficheiro `Lexer.hs` que pode ser compilado com o resto do projeto.

$$\text{Lexer.x} \xrightarrow{\text{alex}} \text{Lexer.hs} \xrightarrow{\text{compilador Haskell}} \text{executável}$$

### Exemplo resolvido

```
{
module Lexer where
}
%wrapper "basic"
$alpha = [_a-zA-Z]
$digit = [0-9]

tokens :-
$white+                ; -- ignorar caracteres brancos
if                      { \_ -> IF }
$alpha($alpha|$digit)*  { \s -> ID s }
$digit+                { \s -> NUM (read s) }
$digit+"."$digit+       { \s -> REAL (read s) }

{
data Token = IF | ID String | NUM Int | REAL Double
}
```

As partes entre `{ }` são código Haskell; o resto são diretivas, definições e padrões. Cada regra tem a forma **padrão** `{ ação }`, onde o padrão é uma expressão regular e a ação é uma função `String -> Token`. Usar: `Lexer.hs` exporta `alexScanTokens :: String -> [Token]`.

```
module Main where
import Lexer
main = do
    txt <- getContents
    print (alexScanTokens txt)
```

O exemplo acima usa a interface `%wrapper "basic"`. Existe também `%wrapper "posn"`, que associa a cada token a sua **posição** (linha e coluna) — útil para reportar erros com precisão. Nesse caso, cada ação passa a ter tipo `AlexPosn -> String -> Token`, com data `AlexPosn = AlexPn Int Int Int` (posição absoluta, linha, coluna), e o padrão de cada regra recebe também `pos` (ex: `\pos s -> ID pos s`).

## Flex (gerador para C)

### Definição — Flex

O **Flex** é o equivalente do Alex mas para C. Recebe um ficheiro `.l` (ex: `lexer.l`) e produz `lex.yy.c`, compilável com o resto do projeto C.

$$\text{ficheiro.l} \xrightarrow{\text{flex}} \text{lex.yy.c} \xrightarrow{\text{compilador C}} \text{executável}$$

## Exemplo resolvido

```
%{
#include "tokens.h"
%}
%option noyywrap
alpha      [_a-zA-Z]
digit      [0-9]
%%

[ \t\n\r]+      /* skip whitespace */
if               { return IF; }
{alpha}{alpha}|{digit}* { return ID; }
{digit}+        { yynum = atoi(yytext); return NUM; }
{digit}+\".\"{digit}+  { yyreal = atof(yytext); return REAL; }
<<EOF>>         { return EOF; /* end of input */ }
```

Estrutura geral de um ficheiro Flex: `%{` declarações `%}`, depois definições auxiliares, `%%`, depois as regras padrão `{ ação em C }`, e opcionalmente mais `%%` seguido de código C auxiliar.

```
#include "tokens.h"
int main(void) {
    int token;
    while ((token = yylex()) != EOF) {
        switch (token) {
            case IF:   printf("IF ");           break;
            case ID:   printf("ID(%s) ", yytext); break;
            case NUM:  printf("NUM(%d) ", yynum); break;
            case REAL: printf("REAL(%f) ", yyreal); break;
        }
    }
}
```

A função `yylex()` devolve o próximo token (um inteiro, definido num ficheiro *header*). A variável global `yytext` contém sempre o texto do token atual — é assim que se acede ao valor (ex: o nome de um identificador). Os valores de tokens não-textuais (como `NUM`, `REAL`) costumam guardar-se em variáveis de estado globais (`yynum`, `yyreal`).

## Atenção

Nota adicional: por omissão o Flex **não** guarda a posição de cada token. Para isso usa-se `%option yylineno`, que mantém a variável global `yylineno` com o número da linha atual — mas é **responsabilidade do programador** ir guardando essa posição à medida que os tokens são consumidos (ao contrário do Alex com `%wrapper "posn"`, que já entrega a posição em cada ação).

## Longest match e first match

## Definição — Regras de desambiguação

Tanto o Alex como o Flex (e geradores semelhantes) usam duas regras, por esta ordem, para decidir qual regra aplicar quando várias expressões regulares poderiam casar com o *input*:

1. **Longest match**: escolhe-se a regra que consome o **maior número de caracteres** de entrada.
2. **First match**: em caso de empate no comprimento, escolhe-se a regra que aparece **primeiro** no ficheiro de descrição.

**Frequente em exame**

Consequência prática (e fonte comum de bugs/perguntas de exame): identificadores e palavras reservadas partilham o mesmo texto possível (`if` casa tanto com o padrão literal `if` como com o padrão de identificador `[_a-zA-Z][_a-zA-Z0-9]*`, com o **mesmo comprimento** — 2 caracteres). Por *first match*, a regra que resolve a ambiguidade corretamente é **colocar os padrões mais específicos (palavras reservadas) antes dos mais gerais (identificadores)** no ficheiro `.x/ .l`. Se a ordem for trocada, `if` seria sempre lido como ID, nunca como IF.

**Comparação rápida Alex vs. Flex**

|                          | Alex                                                | Flex                                    |
|--------------------------|-----------------------------------------------------|-----------------------------------------|
| Linguagem gerada         | Haskell ( <code>.hs</code> )                        | C ( <code>lex.yy.c</code> )             |
| Ficheiro de entrada      | <code>.x</code>                                     | <code>.l</code>                         |
| Acesso ao texto do token | argumento da ação ( <code>\s -&gt; ...</code> )     | variável global <code>yytext</code>     |
| Posição do token         | <code>%wrapper "posn"</code> (automático, por ação) | <code>%option yylineno</code> (manual)  |
| Função de arranque       | <code>alexScanTokens :: String -&gt; [Token]</code> | <code>yylex()</code> (chamado em ciclo) |

**Ligação com a prática (Folha laboratorial 2 — `scanner-c`)**

O laboratório desta semana (`Semana_1/scanner-c/`) dá um analisador lexical em C incompleto (`CLexer.c`) para os tokens ID, NUM, LPAREN, RPAREN, COMMA, IF, e pede para o completar/re-implementar:

- **Exercício 1** (estender `CLexer.c`): usa diretamente a secção “Implementação direta de um DFA em C” acima — o `switch/while` do ficheiro fornecido segue exatamente o padrão de “ler carácter, decidir por casos” descrito ali, só que sem tabela explícita de transições. Repara em particular no aviso da Aula 2 sobre a ordem `if` vs. ID (a alínea sobre `_` como identificador é um caso de expressão regular `[_a-zA-Z]` mal transcrita para o `switch` do C — falta o `case '_'`: a par de `isalpha(c)`).
- **Exercício 2** (expressões regulares, papel e lápis): usa a secção “Expressões regulares” — em particular a caixa de atenção sobre comentários multi-linha explica a técnica geral necessária para a alínea (c).
- **Exercício 3** (reimplementar com `flex`): usa diretamente a secção “Flex” acima, incluindo a estrutura do ficheiro `.l` e o exemplo completo.

## Aula 4 — Análise sintática e gramáticas independentes de contexto

As Aulas 2–3 cobriram a **análise lexical**: transformar texto em bruto numa sequência de *tokens*. Esta aula avança para o passo seguinte do *frontend* (ver o diagrama de fases na Aula 1, secção “Fases de um compilador” — a análise sintática é a fase logo a seguir à lexical): a **análise sintática**, que verifica se a sequência de tokens respeita a estrutura gramatical da linguagem e constrói uma **árvore sintática**.

### O que é a análise sintática

#### Definição — Sintaxe

*Sintaxe*: “estudo das regras e dos princípios que regem a organização dos constituintes das frases” (Dicionário Priberam). Aplicado a uma linguagem de programação: um programa é **sintaticamente bem formado** se respeitar regras de estrutura — chavetas { } e parêntesis ( ) “casados”, operadores com o número correto de operandos, instruções terminadas ou separadas corretamente (;), etc.

Tal como na linguagem natural, um programa pode estar sintaticamente correto e ainda assim não fazer sentido — o exemplo clássico de Chomsky (1957) é a frase em inglês “*Colorless green ideas sleep furiously*” (gramaticalmente válida, semanticamente absurda). Verificar o “faz sentido” é trabalho da **análise semântica** (fase seguinte, fora do âmbito desta aula); a análise sintática só verifica estrutura.

O **analisador sintático** (*parser*) constrói a árvore sintática a partir da sequência de tokens (ou reporta um erro, se a sequência não respeitar nenhuma estrutura válida). A ferramenta matemática usada para especificar essa estrutura é a **gramática independente de contexto**.

### Gramáticas independentes de contexto

#### Definição — Gramática independente de contexto (CFG)

Uma **gramática independente de contexto** é um quádruplo  $G = (\Sigma, N, S, P)$ :

- $\Sigma$  — conjunto de símbolos **terminais** (os tokens — os mesmos que saem do analisador lexical);
- $N$  — conjunto de símbolos **não-terminais** (categorias sintáticas abstratas, ex: “expressão”, “instrução”);
- $S \in N$  — símbolo **inicial**;
- $P$  — conjunto de **produções** da forma  $X \rightarrow \alpha$ , onde  $X$  é um não-terminal e  $\alpha$  é uma sequência (possivelmente vazia) de terminais e/ou não-terminais.

Chama-se “independente de contexto” porque o lado esquerdo de cada produção é **sempre um único não-terminal isolado** (nunca depende do que está à sua volta na frase para saber se a produção se pode aplicar — ao contrário de gramáticas mais gerais, fora do âmbito deste curso).

**Exemplo resolvido**

Gramática usada como fio condutor ao longo desta secção (é a gramática usada nos slides para introduzir derivações, árvores e ambiguidade — vai reaparecer várias vezes abaixo, sempre com a mesma numeração de produções):

$$\Sigma = \{a, b\} \quad N = \{S, B\} \quad \text{inicial: } S$$

Produções:

| Nº  | Produção                    |
|-----|-----------------------------|
| (1) | $S \rightarrow aSB$         |
| (2) | $S \rightarrow \varepsilon$ |
| (3) | $S \rightarrow B$           |
| (4) | $B \rightarrow Bb$          |
| (5) | $B \rightarrow b$           |

**Derivações****Definição — Derivação**

A relação de **derivação**  $\Rightarrow$  substitui um não-terminal pelo lado direito de alguma produção que o tenha como lado esquerdo.  $\Rightarrow^*$  (fecho transitivo) representa zero ou mais passos de derivação.

**Exemplo resolvido**

Derivação completa de  $aabbb$  a partir de  $S$ , usando a gramática acima, anotando cada passo com o número da produção aplicada:

$$S \xRightarrow{1} aSB \xRightarrow{1} aaSBB \xRightarrow{2} aaBB \xRightarrow{4} aaBbB \xRightarrow{5} aabbB \xRightarrow{5} aabbb$$

Passo a passo: (1)  $S \rightarrow aSB$  substitui o  $S$  inicial  $\rightarrow aSB$ . (1) novamente, no  $S$  que sobrou  $\rightarrow aaSBB$  (repara que a produção acrescenta sempre um  $a$  à esquerda e um  $B$  à direita). (2)  $S \rightarrow \varepsilon$  faz o  $S$  desaparecer  $\rightarrow aaBB$  (dois  $a$ 's, dois  $B$ 's por resolver). (4)  $B \rightarrow Bb$  no primeiro  $B \rightarrow aaBbB$  (esse  $B$  vira  $Bb$ , ou seja um  $B$  novo seguido de  $b$ ). (5)  $B \rightarrow b$  no  $B$  que sobrou do passo anterior  $\rightarrow aabbB$ . (5) outra vez, no último  $B \rightarrow aabbb$ . Todos os não-terminais foram eliminados:  $aabbb$  é uma palavra da linguagem.

**Linguagem descrita por uma gramática****Definição —  $L(G)$** 

Partindo do símbolo inicial e substituindo não-terminais pelas produções até só restarem terminais, obtemos uma **palavra** descrita pela gramática. Formalmente, para  $G = (\Sigma, N, S, P)$ :

$$L(G) = \{ w \in \Sigma^* : S \Rightarrow^* w \}$$

**Exemplo resolvido**

**Exercício dos slides, resolvido por completo:** que linguagem descreve a gramática do exemplo acima ( $S \rightarrow aSB \mid \varepsilon \mid B$ ;  $B \rightarrow Bb \mid b$ )? Onde podem ocorrer a's e b's numa palavra aceite, e qual a relação entre o número de a's e de b's?

Primeiro, o que faz cada não-terminal:

- $B$  só gera seqüências de **um ou mais b's**:  $B \rightarrow b$  dá b;  $B \rightarrow Bb$  acrescenta sempre mais um b à direita de um  $B$  já formado. Logo  $B$  gera exatamente a linguagem  $b^+$  (nunca  $b^0$ , porque não há produção  $B \rightarrow \varepsilon$ ).
- $S \rightarrow aSB$  aplicada  $n$  vezes seguidas produz  $a^n S B_1 B_2 \dots B_n$  ( $n$  cópias de  $B$ , cada uma independente das outras — ver a derivação acima, onde duas aplicações de (1) geram exatamente dois  $B$ 's pendentes). Cada  $B_i$  resolve-se depois, independentemente, num  $b^{k_i}$  com  $k_i \geq 1$ .
- O  $S$  que sobra no meio tem de terminar de duas formas possíveis:
  - via (2)  $S \rightarrow \varepsilon$ : não acrescenta mais nada. Com  $n$  cópias de  $B$  já geradas, o total de b's é  $m = \sum_{i=1}^n k_i$ , e como cada  $k_i \geq 1$ , temos  $m \geq n$  (com  $m = n$  exatamente quando todos os  $k_i = 1$ , e  $m$  pode crescer arbitrariamente aumentando qualquer  $k_i$  — logo **todos** os valores  $m \geq n$  são atingíveis). Caso particular  $n = 0$ : dá a palavra vazia.
  - via (3)  $S \rightarrow B$ : acrescenta mais **um**  $B$  extra (o  $(n+1)$ -ésimo), que por si só já gera  $b^k$  com  $k \geq 1$ . Isto dá  $m \geq n + 1$  — mas este intervalo já está contido no anterior (para o mesmo  $n$ , a via (2) já atinge qualquer  $m \geq n$ , incluindo  $m = n + 1, n + 2, \dots$ ), por isso não acrescenta palavras novas à linguagem, só dá origem a **derivações alternativas** para palavras que já eram atingíveis — é precisamente esta segunda via que torna a gramática **ambígua** (ver secção seguinte).

Juntando tudo:  $n$  pode ser qualquer natural  $\geq 0$ , e para cada  $n$ ,  $m$  (número de b's) pode ser qualquer natural  $\geq n$ . Logo:

$$L(G) = \{ a^n b^m : n \geq 0, m \geq n \}$$

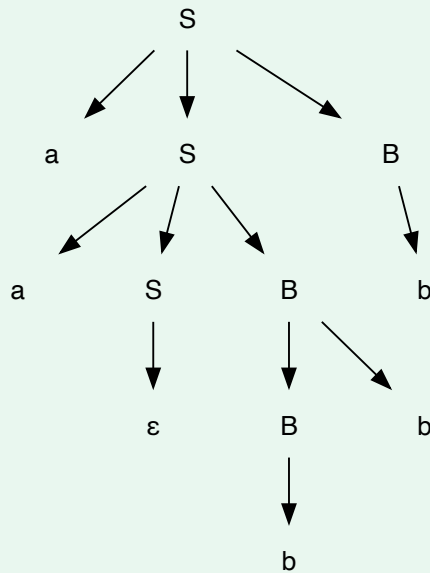
Em palavras (a resposta pedida “numa frase” pelo exercício): **a linguagem das palavras formadas por zero ou mais a's seguidos de zero ou mais b's, em que o número de b's nunca é inferior ao número de a's** (incluindo a palavra vazia, quando  $n = m = 0$ ). Todos os a's aparecem sempre à esquerda de todos os b's — a gramática nunca permite intercalar os dois símbolos.

**Árvores sintáticas****Definição — Árvore sintática**

Cada passo de uma derivação pode representar-se como um nó numa **árvore sintática**: uma produção  $X \rightarrow \alpha_1 \dots \alpha_n$  corresponde a um nó  $X$  com  $n$  sub-árvores, uma por cada símbolo de  $\alpha_1, \dots, \alpha_n$  (por ordem, da esquerda para a direita).

**Exemplo resolvido**

Árvore sintática correspondente à derivação de  $aabbb$  feita acima ( $S \xRightarrow{1} aSB \xRightarrow{1} aaSBB \xRightarrow{2} aaBB \xRightarrow{4} aaBbB \xRightarrow{5} aabbB \xRightarrow{5} aabbb$ ):



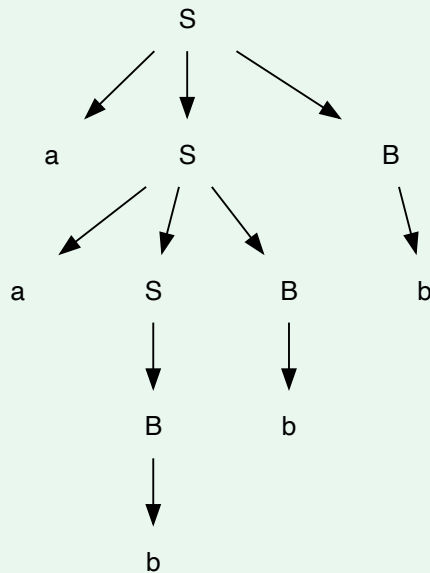
Lendo a árvore de cima para baixo: a raiz  $S$  usa a produção (1) e tem três filhos —  $a$ , o  $S$  do meio, e o  $B$  mais à direita (o “ $B$  exterior”). O  $S$  do meio usa (1) outra vez — três filhos:  $a$ , o  $S$  mais interno, e um segundo  $B$  (o “ $B$  do meio”). O  $S$  mais interno usa (2) e resolve-se em  $\varepsilon$  (fim da recursão). O “ $B$  do meio” usa (4)  $B \rightarrow Bb$  — dois filhos: um novo  $B$ , e  $b$ ; esse novo  $B$  usa (5)  $B \rightarrow b$ . O “ $B$  exterior” usa diretamente (5)  $B \rightarrow b$ . Lendo as folhas da esquerda para a direita:  $a$ ,  $a$ , (nada, do  $\varepsilon$ ),  $b$  (do  $B$  novo dentro do  $B$  do meio),  $b$  (do próprio  $B$  do meio),  $b$  (do  $B$  exterior) — dá exatamente  $aabbb$ .

**Gramáticas ambíguas****Definição — Ambiguidade**

Uma gramática diz-se **ambígua** se existe pelo menos uma palavra da sua linguagem que admite **duas (ou mais) árvores sintáticas distintas**. Nota: derivações diferentes podem corresponder à **mesma** árvore (diferindo só na ordem em que os não-terminais são substituídos) — o que importa para a ambiguidade é a árvore, não a derivação.

## Exemplo resolvido

A mesma gramática do exemplo acima é **ambígua**: a palavra **aabbb** também se obtém pela derivação alternativa  $S \xRightarrow{1} aSB \xRightarrow{1} aaSBB \xRightarrow{3} aaBBB \xRightarrow{5} aabBB \xRightarrow{5} aabbB \xRightarrow{5} aabbb$  — que usa (3)  $S \rightarrow B$  no  $S$  mais interno (em vez de (2)  $S \rightarrow \varepsilon$ ), dando **três**  $B$ 's independentes, cada um resolvido diretamente por (5)  $B \rightarrow b$ , sem nenhum usar (4)  $B \rightarrow Bb$ :



Esta árvore é **estruturalmente diferente** da anterior (a primeira tinha um  $B$  que se expandia em  $Bb$ ; esta tem três  $B$ 's todos “folha única” via (5)) apesar de produzir a mesma palavra **aabbb** — por definição, isto basta para a gramática ser ambígua. Esta é exatamente a razão, identificada na secção anterior, por que a via (3)  $S \rightarrow B$  não alarga a linguagem: só dá um caminho alternativo para palavras já alcançáveis por (2).

## Frequente em exame

**Porque é que a ambiguidade importa?** Como modelo matemático para *descrever* uma linguagem, uma gramática ambígua é perfeitamente válida. Mas num compilador a gramática também serve para **atribuir significado** aos fragmentos do programa (é a árvore sintática que a análise semântica e a geração de código percorrem depois) — se a mesma palavra tem duas árvores possíveis, o compilador não sabe qual significado escolher. Por isso as linguagens de programação são desenhadas para que a sua gramática não seja ambígua (ou, quando é impraticável evitar completamente, para que existam regras explícitas de desambiguação — ver “dangling else” abaixo). Este é um tema clássico de exame: identificar que uma gramática é ambígua exibindo **duas árvores diferentes** para a mesma palavra (não basta exibir duas derivações — têm de corresponder a árvores diferentes).

## Exemplo central: expressões aritméticas

### Exemplo resolvido

Gramática de expressões aritméticas (não-terminal  $E$ , terminais  $\text{num}, +, *, (, )$ ):

$$E \rightarrow E + E \mid E * E \mid \text{num} \mid (E)$$

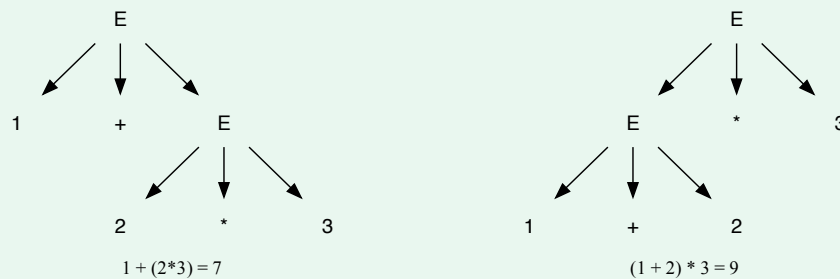
Esta gramática é **ambígua** — a palavra  $1+2+3$  admite duas árvores:



Ambas as árvores calculam o mesmo resultado (6), mas são **árvores diferentes** — isso já basta para a gramática ser ambígua, mesmo sem nenhuma consequência prática neste caso particular.

### Exemplo resolvido

A palavra  $1+2*3$  já revela o problema **na prática**: as duas árvores possíveis dão **resultados diferentes**.



Se um compilador usasse esta gramática tal como está, a árvore que calhasse ser construída determinaria se  $1+2*3$  valeria 7 ou 9 — um comportamento imprevisível e obviamente inaceitável para uma linguagem de programação.

## Eliminar ambiguidade: associatividade e precedência

Frequentemente consegue-se eliminar a ambiguidade **reescrevendo a gramática** (sem mudar a linguagem que ela descreve). Para expressões aritméticas há duas escolhas a fixar explicitamente:

- **Associatividade** dos operadores:  $1+2+3$  interpretado à esquerda  $((1+2)+3)$  ou à direita  $(1+(2+3))$ ?
- **Prioridade (precedência)** entre operadores:  $1+2*3$  interpretado como  $1 + (2*3)$  ou como  $(1+2)*3$ ?

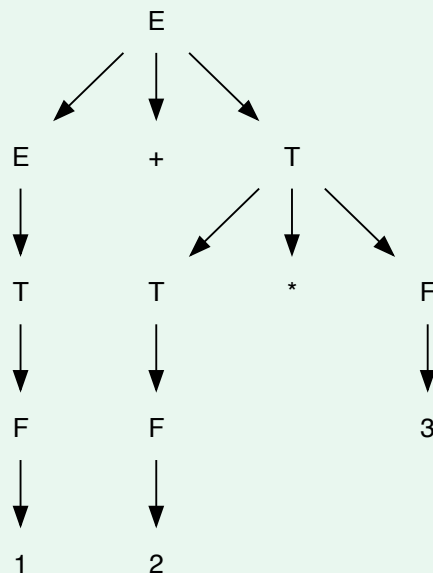
**Definição — Gramática de expressões desambiguada**

$$E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow \text{num} \mid (E)$$

Novos não-terminais e ideia da construção: uma **expressão** ( $E$ ) é uma soma de **termos** ( $T$ ); um **termo** é um produto de **fatores** ( $F$ ); um **fator** é uma constante ou uma expressão entre parêntesis. As produções de  $E$  e  $T$  têm **recursão à esquerda** ( $E \rightarrow E + T$ , não  $E \rightarrow T + E$ ) — é isso que força **associatividade à esquerda** para  $+$  e  $*$ . E como só se pode chegar a um  $*$  **através** de um  $T$  (nunca diretamente a partir de  $E$ ), um  $*$  “agarra” sempre o seu operando antes de ele poder ser usado numa soma — é isso que dá **precedência maior** ao  $*$  sobre o  $+$ .

**Exemplo resolvido**

Com a gramática desambiguada,  $1+2*3$  só tem **uma** árvore possível:



$E$  (raiz) usa  $E \rightarrow E + T$ : filho esquerdo  $E \rightarrow T \rightarrow F \rightarrow 1$ ; filho direito  $T$ , que por sua vez usa  $T \rightarrow T * F$ :  $T \rightarrow F \rightarrow 2$  à esquerda do  $*$ ,  $F \rightarrow 3$  à direita. O  $*$  fica “preso” dentro do ramo direito do  $+$ , antes de esse ramo poder combinar-se com o 1 — por isso o resultado é sempre  $1 + (2 \times 3) = 7$ , nunca  $(1 + 2) \times 3$ .

**Atenção**

**Exercício dos slides (para tentar sozinho, técnica análoga à acima):** uma gramática de “programas sequenciais” com  $S \rightarrow S; S \mid \text{ident} = E \mid \text{ident}++$  e  $E \rightarrow \text{ident} \mid \text{num} \mid E + E$  é ambígua em **dois** sítios distintos — não só nas expressões ( $E$ , mesmo problema do  $E \rightarrow E + E$  acima, mesma técnica de correção com precedência/associatividade), mas também nas próprias instruções ( $S \rightarrow S; S$  tem a mesma forma estrutural que  $E \rightarrow E + E$ , logo sofre do mesmo tipo de ambiguidade de associatividade — a correção é a técnica de recursão à esquerda usada acima para  $E \rightarrow E + T \mid T$ , aplicada agora a  $S$ ).

**O problema do “dangling else”**

Muitas linguagens de programação permitem `if/then` com `else` opcional:

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid \dots$$

### Atenção — Dangling else

Esta gramática é ambígua de uma forma particular: em

`if e1 then if e2 then s1 else s2`

o `else s2` pode ligar-se ao `if e1` ou ao `if e2` — duas interpretações válidas segundo a gramática:

- `if e1 then { if e2 then s1 else s2 }` (else liga ao if mais próximo)
- `if e1 then { if e2 then s1 } else s2` (else liga ao if mais afastado)

Normalmente prefere-se a primeira: **o else associa-se sempre ao if mais próximo** (a convenção universal na maioria das linguagens).

### Frequente em exame

**Resolução formal**, para o caso de se preferir corrigir a gramática em vez de tratar isto no analisador: introduzir dois não-terminais,  $M$  (*matched statements* — instruções onde todo `if` já tem o seu `else`) e  $U$  (*unmatched* — instruções com um `if` ainda por fechar):

$$S \rightarrow M \mid U$$

$$M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid \dots$$

$$U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U$$

A ideia: um `if...then...else...` só pode ser  $M$  (fechado) se **ambos** os ramos forem  $M$  — isto impede que o `then` interno fique “por fechar” dentro de um `if...else` já completo, forçando o `else` externo a “descer” e agarrar o `if` mais próximo disponível. Na prática, contudo, **é frequente preferir não mexer na gramática** e resolver a ambiguidade diretamente na implementação do analisador sintático (técnica vista em aulas seguintes).